

Week 2 Resources: From Intuition to REINFORCE

Goal of Week 2

Week 1 focused on intuition: understanding agents, environments, states, actions, rewards, and policies, and observing simple environments like CartPole with random and hand-written controllers. Week 2 takes the next step: you will train your first learning agent using the REINFORCE policy gradient algorithm on CartPole-v1.

By the end of this week, you should be able to:

- Explain the idea of a parameterized policy $\pi_{\theta}(a | s)$.
- Describe what a policy gradient is trying to optimize.
- Sketch the REINFORCE update at a high level.
- Interpret simple learning curves for an RL agent.

Recap: Key Ideas from Week 1

Before diving into REINFORCE, make sure these concepts are clear:

- **Agent:** the decision-maker (e.g., your CartPole controller).
- **Environment:** everything the agent interacts with but does not control (e.g., the CartPole simulator).
- **State:** a representation of “what is going on now” (e.g., cart position, velocity, pole angle, pole angular velocity).
- **Action:** what the agent chooses to do (e.g., push left or right).
- **Reward:** a scalar feedback signal that the agent tries to maximize over time.

You also saw that a policy is simply a rule for choosing actions given states, and that random or hand-coded policies have clear limitations.

From Hand-Coded Rules to Parametric Policies

In Week 1, you might have used a rule like:

“If the pole is leaning right, push right; otherwise push left.”

This is a deterministic policy: the same state always leads to the same action. In Week 2, we replace this with a *parametric* policy $\pi_\theta(a \mid s)$, often a small neural network with parameters θ .

For CartPole with two actions, the policy network outputs two numbers that we interpret as probabilities:

$$\pi_\theta(a = 0 \mid s) + \pi_\theta(a = 1 \mid s) = 1.$$

Sampling from this distribution introduces stochasticity, which is useful for exploration.

The Objective: Maximize Expected Return

For an episodic task like CartPole, we define the return from time step t as:

$$G_t = \sum_{k=t}^{T-1} \gamma^{k-t} r_k,$$

where $\gamma \in [0, 1]$ is the discount factor, r_k is the reward at time k , and T is the end of the episode.

The goal of the agent is to maximize the expected return:

$$J(\theta) = \mathbb{E}_{\pi_\theta}[R],$$

where R is the total (possibly discounted) return of an episode generated by following π_θ .

The Policy Gradient Idea

Policy gradient methods adjust θ in the direction that increases $J(\theta)$:

$$\nabla_\theta J(\theta) \approx \mathbb{E} \left[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t) G_t \right].$$

Intuitively:

- If a sequence of actions leads to high return, we want to increase the probability of those actions in the states where they were taken.
- If a sequence of actions leads to low return, we want to decrease those probabilities.

The log-probability $\log \pi_\theta(a_t \mid s_t)$ is used because it turns the gradient of a product of probabilities into a sum, which is easier to estimate.

REINFORCE in One Sentence

The REINFORCE algorithm can be summarized as:

“Sample episodes from the current policy, compute returns for each time step, and adjust the policy parameters to increase the log-probability of actions that led to higher-than-average returns.”

In practice, this leads to a loss of the form:

$$L(\theta) = - \sum_t \log \pi_{\theta}(a_t | s_t) \hat{G}_t,$$

where $\hat{G}_t$ is some estimate of return (possibly normalized or with a baseline subtracted).

Variance Reduction and Baselines

A common issue with naive REINFORCE is high variance in gradient estimates. One simple trick is to subtract a baseline b from the return without changing the expectation of the gradient:

$$\sum_t \log \pi_{\theta}(a_t | s_t) (G_t - b).$$

A typical choice is the mean return over an episode batch. In the assignment, you will normalize returns across a trajectory, which plays a similar stabilizing role.

Connecting Back to Week 1

As you implement REINFORCE, keep asking yourself:

- **What is the agent?** The policy network selecting actions.
- **What is the environment?** The CartPole simulator you interact with through Gymnasium.
- **What is the state?** The observation vector (cart position, velocity, pole angle, angular velocity).
- **What are the actions?** Discrete choices: push left or push right.
- **What is the reward?** Typically +1 per time step until failure in CartPole.

If you can answer these five questions for any RL problem, you have a solid conceptual foundation.

Suggested External Resources (Optional)

You only need this document and the assignment to complete Week 2, but if you want more perspective:

- A short policy gradient section from an introductory RL blog or course notes.
- A high-level video explaining policy gradients or REINFORCE without heavy math.
- The policy gradient section of an open-source RL course (e.g., focusing on code over derivations).

Use external resources to reinforce your understanding, not as something to memorize. The goal is intuition and the ability to read and write simple policy gradient code.

Self-Check Before Moving On

You are ready for deeper algorithms (like PPO and multi-agent setups) when you can:

- Explain in words what REINFORCE does with the data from an episode.
- Draw a simple diagram of the training loop: collect episodes, compute returns, compute loss, take an optimizer step.
- Look at a learning curve and tell whether the agent is improving or stuck.